VRIFA: An Open Reference Implementation for Resin Flow-Front Detection in VARTM Process Video

J.C. Vaught[1]
University of South Carolina, Columbia, SC, 29208, USA

I. Introduction

II. Related Work

III. Method

A. Pipeline overview

The detection algorithm treats each video frame as a comparison against a quiescent reference image of the dry preform. Where resin has wetted the fabric, the local appearance darkens, and the absolute change is largest near the advancing flow front. The pipeline computes that per-pixel difference inside a rectangular region of interest, clips it to admit only darkening so that specular highlights from the vacuum bag are rejected, normalizes the resulting field, thresholds it into a binary candidate mask, cleans the mask with morphological closing and opening, removes small connected components, and finally locks pixels whose wet label has persisted for several consecutive frames. The locked mask is the canonical output, the optional heatmap renders the underlying delta field for visual diagnosis, and the optional overlay draws the mask boundary onto the original Resin Vacuum Assisted Transfer Molding (VARTM) frame for human review. Figure Fig. 1 shows the twelve stages in order.

Pipeline diagram placeholder. Twelve stages, left-to-right. Decode, colorspace, region of interest, peak update, reference selection, delta, blur, normalization, threshold, morphology, locking, render.

Fig. 1 Twelve-stage VRIFA pipeline. Each frame flows from decode through an appearance-difference comparison against a chosen reference, into a thresholded and morphologically cleaned mask, and finally through a temporal lock that stabilizes the boundary across frames. The same mask drives the binary, heatmap, and overlay renderers.

The pipeline is deliberately classical, which is what makes it suitable as a reference implementation for downstream learning systems. Every stage is an explicit function of one frame, the chosen reference image, and a small set of scalar parameters, so the output is reproducible bit-for-bit at the stage boundary and the algorithm can be audited end-to-end without recourse to a black-box model.

B. Frame decode and colorspace conversion

The first stage decodes each Blue-Green-Red (BGR) frame from the input video. The second converts that frame into a working colorspace. VRIFA exposes four options, namely the International Commission on Illumination (CIE) 1976 *L*a*b** colorspace (CIELAB), Red-Green-Blue (RGB), Hue-Saturation-Value (HSV), and 8-bit grayscale, all

[1]Graduate Research Assistant, Department of Mechanical Engineering, AIAA Student Member.

routed through OpenCV color converters. CIELAB is the default because resin wetting darkens the lightness channel L^* much more than it shifts chrominance, which makes the single-channel L^* projection in stage six both sufficient and robust. We denote the converted frame at index t by $F_t \in \mathbb{R}^{H\times W\times C}$, where C is the channel count of the chosen colorspace.

C. Region-of-interest mask

A rectangular region of interest (ROI) excludes the part frame, manifold flange, and bag wrinkles outside the laminate. Given fractional margins $m_{\text{top}}, m_{\text{bot}}, m_{\text{left}}, m_{\text{right}}$ each clamped to $[0, 0.49]$, the binary mask $R \in \{0,1\}^{H\times W}$ is

$$R(y,x) = \begin{cases} 1 & \text{if } \lfloor m_{\text{top}} H \rfloor \le y < H - \lfloor m_{\text{bot}} H \rfloor \text{ and } \lfloor m_{\text{left}} W \rfloor \le x < W - \lfloor m_{\text{right}} W \rfloor \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

A single CLI flag `--roi-margin` sets all four margins symmetrically, with per-edge overrides available. All subsequent stages operate only on pixels inside R.

D. Peak-brightness map

VRIFA accumulates a running maximum of the working channel across all frames seen so far, denoted $P_t \in \mathbb{R}^{H\times W}$. After conversion of frame F_t, the map is updated pixelwise as

$$P_t(y,x) = \max(P_{t-1}(y,x), F_t(y,x,0)), \tag{2}$$

with $P_0(y,x) = F_0(y,x,0)$. The motivation is that the dry preform is, by definition, the brightest the fabric will ever appear in the working channel. Treating P_t as the local reference instead of a single early frame absorbs slow lighting drift caused by lamp warm-up and bag deformation. The peak map can be disabled at the command line with `--no-peak-reference`.

E. Reference selection

The reference image G_t used for the difference computation has five modes. *First* pins $G_t = F_0$ for every t. *Running* updates an exponential moving average $G_t = (1-\alpha)G_{t-1} + \alpha F_t$ with default $\alpha = 0.05$. *Prev* uses a fixed-offset history, $G_t = F_{t-k}$, with the buffer bootstrapped from F_0 until $t > k$. *Absolute* pins G_t to a user-specified absolute frame index. *Dynamic* adapts the lag online from a sqrt-area growth model fit to the early frames. The default mode is *first*, which, combined with the peak map of Eq. Eq. (2), gives a piecewise-static reference whose only adaptation is the peak update.

The dynamic mode is worth a closer look because it is what enables comparable behavior across runs of different fill speeds. For the first N_{calib} frames after detection bootstrapping, VRIFA records the wet area a_t inside the ROI and the elapsed time $\tau_t = \frac{t-1}{f}$ in seconds, where f is the video frame rate. It then estimates the growth-rate factor

$$\kappa = \text{median}_{t:\tau_t>0, a_t>0}\left(\frac{a_t}{\tau_t^{1.5}}\right). \tag{3}$$

The exponent 1.5 comes from the area-growth law observed for radial Darcy infusion, where wetted area is approximately proportional to time at the three-halves power once flow is well established. Given κ and a target wet fraction $\rho \in [0,1]$ (default 0.2) of $|R|$ ROI pixels, the dynamic lag $\Delta\tau_t$ that places the reference at the moment when the wet area was a fraction ρ of the current area is

$$\Delta\tau_t = \lambda \cdot \left[\left(\frac{\rho\ |R|}{\kappa} + \sqrt{\tau_t}\right)^2 - \tau_t\right], \tag{4}$$

clipped to be non-negative and scaled by a user lag factor λ (default 1.0). The reference frame is then read from a small cache at index $\max(1, \lfloor(\tau_t - \Delta\tau_t)f\rfloor + 1)$, falling back to the first frame whenever the calibration has not yet produced a finite κ. A linear-mode override is available for diagnostic comparisons and is configured by `--dynamic-lag-linear`, `--dynamic-lag-linear-start`, and `--dynamic-lag-linear-max`.

F. Delta computation

Stage six produces the per-pixel scalar field $D_t \in \mathbb{R}_{\geq 0}^{H \times W}$ that drives all downstream decisions. Two modes are supported. The default *darken-only* mode operates only on the working (first) channel and records how much darker the frame is than its reference,

$$D_t(y,x) = R(y,x) \cdot \max(0, w_0 \cdot (G_t^\star(y,x) - F_t(y,x,0))), \tag{5}$$

where $G_t^\star$ equals the peak map P_t if `--peak-reference` is enabled, and otherwise equals the channel-zero slice of the reference G_t. The clip to non-negative values discards every pixel that becomes *brighter* than the reference, which removes specular flashes from the silicone vacuum bag and from condensation, neither of which are wetting events. The full-color mode replaces Eq. Eq. (5) with the weighted Euclidean distance across all channels,

$$D_t(y,x) = R(y,x) \cdot \sqrt{\sum_{c=0}^{C-1} w_c^2 \cdot (F_t(y,x,c) - G_t(y,x,c))^2}, \tag{6}$$

with channel weights w_c defaulting to one. The full-color mode is exposed via `--no-darken-only` and is intended for colorspaces such as HSV where chrominance shifts are diagnostic.

G. Gaussian blur

The delta field is smoothed with a separable Gaussian kernel of size $k_b \times k_b$ to suppress the per-pixel speckle that survives the colorspace conversion. The default kernel size is $k_b = 9$ pixels, forced odd at runtime. Smoothing happens before normalization so that the dynamic range of the normalized image is set by structure rather than by isolated outliers. The blur can be disabled with `--skip-blur`.

H. Normalization to 8-bit

The smoothed delta is rescaled to the byte range using OpenCV min-max normalization,

$$\tilde{D}_t(y,x) = \mathrm{clip}_{[0,255]}\left(255 \cdot \frac{D_t^{\mathrm{blur}}(y,x) - \min D_t^{\mathrm{blur}}}{\max D_t^{\mathrm{blur}} - \min D_t^{\mathrm{blur}}}\right), \tag{7}$$

producing $\tilde{D}_t \in \{0, ..., 255\}^{H \times W}$. The normalized field is what feeds both the threshold-selection stage and the heatmap renderer, so any color visualization the user inspects is exactly the field on which the threshold is computed.

I. Threshold selection

Stage nine converts $\tilde{D}_t$ into a binary candidate mask $B_t \in \{0, 255\}^{H \times W}$. Three thresholding modes share a single offset. If the user passes a manual threshold τ_{man}, the algorithm sets $\tau = \tau_{\mathrm{man}} + \delta_\tau$. If the user passes a percentile $p \in [0, 100]$, the algorithm sorts the ROI-restricted pixels of $\tilde{D}_t$ and uses NumPy-compatible linear interpolation between adjacent ranks to compute the p-th percentile, then adds δ_τ. Otherwise, Otsu's between-class variance method is applied to the full $\tilde{D}_t$ histogram to recover an automatic threshold τ_{otsu}, and the offset is again added. In all three cases the final threshold is clamped to $[0, 255]$,

$$\tau = \mathrm{clip}_{[0,255]}(\tau_{\mathrm{base}} + \delta_\tau), \tag{8}$$

and the binary mask is

$$B_t(y,x) = \begin{cases} 255 & \text{if } \tilde{D}_t(y,x) > \tau \\ 0 & \text{otherwise} \end{cases}. \tag{9}$$

The default threshold offset is $\delta_\tau = -30$, which biases Otsu slightly toward the wet class to capture the partially-wetted halo that classical Otsu would otherwise miss. Manual and percentile modes are reserved for parameter studies and are not used for the headline numbers reported in this paper.

J. Morphology

The binary mask is cleaned by morphological closing followed by opening with a structuring element S of shape s and odd kernel size $k_m \times k_m$. Closing fills small gaps and welds neighboring wet patches into a single front,

$$M_t^{\mathrm{close}} = (B_t \oplus S) \ominus S, \tag{10}$$

and opening removes isolated specks below the kernel size,

$$M_t^{\text{open}} = \big(M_t^{\text{close}} \ominus S\big) \oplus S, \tag{11}$$

with $\oplus$ and $\ominus$ denoting dilation and erosion. Each operation runs for a configurable number of iterations, defaulting to one. The structuring element is *ellipse* by default with optional *rect* and *cross* alternatives, which is exposed because the front shape changes with infusion geometry. After closing and opening, connected-components labeling discards any region whose pixel area is below the threshold $a_{\min}$ pixels (default 400). The kernel parities are forced odd at runtime so that anchor handling is symmetric.

K. Temporal locking

Stage eleven imposes hysteresis along the time axis. For each pixel (y, x), VRIFA maintains a counter $c_t(y, x)$ and a sticky locked-pixel map $L_t(y, x) \in \{0, 255\}$. The counter increments while the pixel is wet in the cleaned mask M_t and resets to zero on any dry frame. Once the counter reaches the parameter n_{lock} frames, the locked map is set to 255 and never decreases thereafter. The output is the elementwise maximum of the cleaned mask and the locked map. Formally,

$$c_t(y,x) = \begin{cases} c_{t-1}(y,x) + 1 & \text{if } M_{t(y,x)} > 0 \\ 0 & \text{otherwise} \end{cases}, \tag{12}$$

$$L_t(y,x) = \max\Big(L_{t-1}(y,x), 255 \cdot \mathbb{1}\big[c_{t(y,x)} \geq n_{\text{lock}}\big]\Big), \tag{13}$$

$$\widehat{M}_t(y,x) = \max(M_t(y,x), L_t(y,x)). \tag{14}$$

The locked mask $\widehat{M}_t$ is the canonical output of the algorithm. Locking is what gives the boundary the temporal stability needed for downstream tasks. A flicker that survives morphology for fewer than n_{lock} frames is treated as a transient and not committed to the locked layer, while a wet pixel that is occluded by a glove or a fold is preserved across the occlusion. The default lock window is $n_{\text{lock}} = 3$ frames; setting `--lock-frames 0` disables the stage altogether.

L. Heatmap and overlay renderers

The last two stages exist for inspection and for label export. The heatmap renderer maps the normalized delta $\tilde{D}_t$ through OpenCV's *turbo* colormap to produce a three-channel pseudocolor image. The overlay renderer extracts the boundary of $\widehat{M}_t$ via a 5×5 rectangular morphological gradient and paints those boundary pixels red on a copy of the original BGR frame. Neither renderer is part of the detection logic. They simply expose, in human-readable form, the field on which the threshold acted and the boundary that the locked mask encloses.

For machine-readable export, contour extraction emits Common Objects in Context (COCO) and YOLO-format polygons for every connected component of $\widehat{M}_t$. Polygon segmentation is computed with `findContours`, optionally simplified by Douglas-Peucker with tolerance ε, and optionally densified to a maximum edge length so that downstream rasterization preserves curvature. The annotation-sampling utility selects which frames receive labels using one of three modes, namely *all*, evenly-spaced *count*, or fixed-*stride*. Mathematically, the *count*-mode index for $i \in \{0, ..., n-1\}$ given a video of T frames is $j_i = \big\lfloor i \frac{T-1}{n-1} \big\rfloor$, with deduplication of consecutive ties, which exactly reproduces NumPy's integer-truncated `linspace` behavior.

M. Hyperparameter defaults

Table Table 1 collects the parameters exposed by the command-line interface together with the values used for every result reported in this paper. Defaults were chosen during development and held fixed across all runs in the evaluation set, so the reported metrics reflect the algorithm as a user would see it on a fresh checkout rather than the outcome of a per-video search.

Table 1 Hyperparameter defaults shipped in `vrifa-cli` and used for every result in this paper. Symbols correspond to the variables introduced in the formalism above; see Eqs. Eq. (1) through Eq. (14).

Symbol / flag	Default	Description
`--colorspace`	`CIELAB`	Working colorspace for the difference computation.
`--roi-margin`	`0.15`	Symmetric fractional ROI margin per edge.
`--ref-mode`	`first`	Reference selection mode.
`--ref-running-alpha`, α	`0.05`	EMA weight for the running reference.
`--peak-reference`	`true`	Use the running peak map in the working channel.
`--darken-only`	`true`	Clip the delta to non-negative wetting deltas.
`--blur-kernel`, k_b	`9`	Gaussian blur kernel size in pixels.
`--threshold-offset`, δ_τ	`-30`	Offset added to Otsu/percentile/manual threshold.
`--morph-kernel`, k_m	`13`	Morphology structuring-element size.
`--morph-shape`	`ellipse`	Structuring-element shape.
`--morph-close-iterations`	`1`	Morphological closing iterations.
`--morph-open-iterations`	`1`	Morphological opening iterations.
`--min-area`, $a_{\min}$	`400`	Minimum connected-component area, in pixels.
`--lock-frames`, n_{lock}	`3`	Temporal lock window, in frames.
`--frame-step`	`1`	Frame stride at decode time.
`--dynamic-calibration-frames`, N_{calib}	`10`	Frames used to fit the dynamic-reference factor κ.
`--dynamic-target-fraction`, ρ	`0.2`	Target wet-area fraction for dynamic-reference lag.
`--dynamic-lag-scale`, λ	`1.0`	Multiplicative scale on the dynamic-mode lag.
`--dynamic-ref-cache-size`	`32`	Frames cached for the dynamic-reference reader.

N. Implementation pointer

VRIFA is organized as a Cargo workspace with five crates. The `vrifa-core` crate hosts the stage logic, with one Rust source file per stage under `crates/vrifa-core/src/`. The file names track the stage names used above, namely `colorspace.rs`, `roi.rs`, `peak.rs`, `reference.rs`, `delta.rs`, `morphology.rs`, `threshold.rs`, `lock.rs`, `heatmap.rs`, `overlay.rs`, `contours.rs`, and `sampling.rs`, with a small `cvutil.rs` shim that converts between `ndarray` arrays and OpenCV `Mat` views. The `vrifa-io` crate wraps video decode and encode through OpenCV's `VideoCapture` and `VideoWriter`, plus an asynchronous PNG writer used for per-frame artifact dumps. The `vrifa-cli` crate is the user-facing binary; it owns argument parsing, run orchestration, the dynamic-reference cache, and the per-frame loop that ties the twelve stages together. The `vrifa-annotations` crate handles COCO and YOLO export of the contours produced by `vrifa-core::contours`. The `vrifa-py` crate exposes the same stage functions as a PyO3 module, which is internal-development tooling rather than a user-facing interface; everything a researcher would call from the command line goes through `vrifa-cli`.

The Implementation section, below, returns to the same crate layout and describes the buffer-reuse strategy, the OpenCV bindings, and the bit-exact stage-parity result that motivates VRIFA being released as a *reference* implementation rather than as one detector among many.

IV. Implementation

V. Datasets and Labeling Protocol

VI. Quantitative Agreement Results